

Name of the Student	KOTHAKOTA RAKESH
Reg. No	192525075
GitHub Link	https://github.com/Rakesh192525075/MLA0403.git

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning
SLOT : A

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

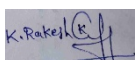
Course Outcome Weightage: 10%
Assessment Tool Weightage: 30%

Duration: 90 minutes
Mark: 25

Code of Conduct:

I A. Santhosh kumar reddy/192525269(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student: KOTHAKOTA RAKESH

Criteria	Problem Understanding & Requirement Analysis (2)	Model Architecture & Design (2.5)	Application of Deep Learning Techniques (2.5)	Model Development & Implementation Strategy (2)	Performance Analysis & Justification (1)	Total (10)
Weightage	20 %	30%	25%	15 %	10 %	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS:**05****Total Marks:25****RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding ; some requirements unclear	Poor understanding ; misinterprets problem context
Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
		solve the problem			
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

ANSWERS

Question 1

Design and develop a CNN-based transfer learning model for a hospital with only 2,000 labeled X-ray images to classify images into Normal and Abnormal categories, using a pre-trained ResNet model.

Answer 1

Approach: Since the hospital has a very small dataset (2,000 images), training a CNN from scratch is not feasible. Transfer learning with a pre-trained ResNet (trained on ImageNet) is used, since the low-level and mid-level features it has already learned (edges, textures, shapes) are generally useful for X-ray images as well.

1. Which layers to freeze: The early and middle convolutional layers of ResNet, which capture generic, low-level features (edges, curves, textures), are frozen so that their learned weights are not updated. This preserves useful general-purpose knowledge and avoids overfitting on a small dataset.
2. Which layer to replace: The final fully connected (classification) layer of ResNet, which was designed for 1,000 ImageNet classes, is removed and replaced with a new fully connected layer having 2 output neurons (Normal and Abnormal), followed by a Softmax activation.
3. How the model should be trained: Only the newly added fully connected layer and the last one or two convolutional blocks are fine-tuned (unfrozen) using a small learning rate, while the frozen layers keep their pre-trained weights. Data augmentation (rotation, flipping, zoom) is applied to artificially expand the small dataset. The model is trained using categorical cross-entropy loss with the Adam optimizer, and validation accuracy is monitored to prevent overfitting (using early stopping / dropout).

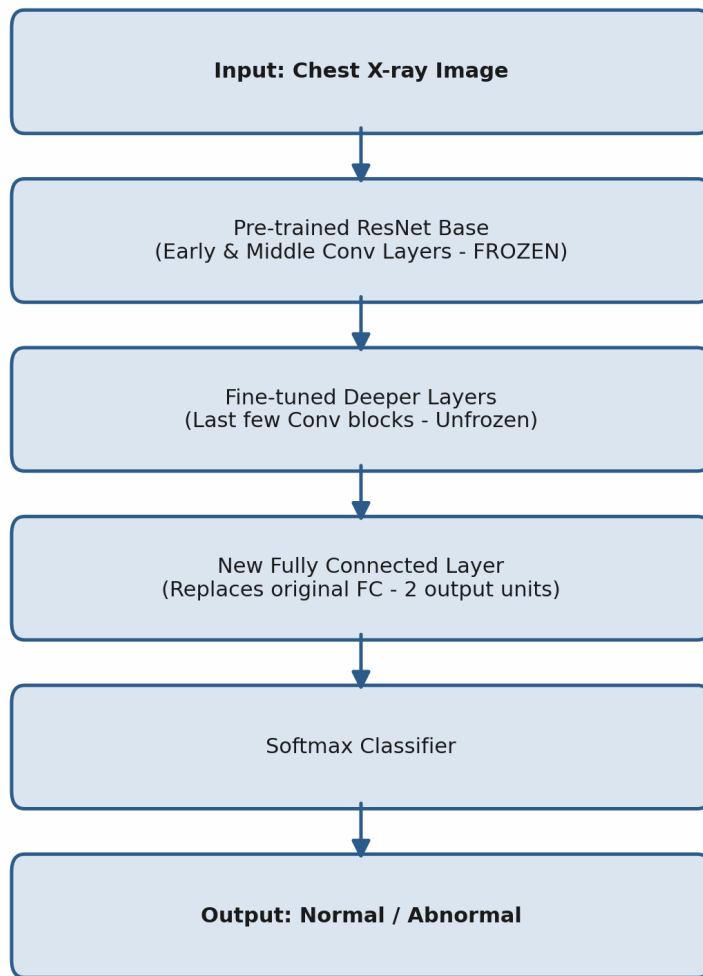


Fig 1: Flowchart - CNN-based Transfer Learning Model (ResNet) for X-ray Classification

Question 2

Design a DenseNet-based classification model to identify five types of plant diseases from leaf images with limited training samples, and explain how dense connections help reuse features.

Answer 2

Approach: A DenseNet (Densely Connected Convolutional Network) architecture is used because it connects each layer to every other layer in a feed-forward manner within a dense block, which is highly effective when only limited training data is available.

1. **Architecture:** The input leaf image first passes through an initial convolution and pooling layer. It then passes through a series of Dense Blocks, each containing multiple convolutional layers. Between dense blocks, Transition Layers (1x1 convolution + pooling) are used to reduce the spatial dimensions and number of feature maps.
2. **How dense connections help:** Inside a Dense Block, each layer receives as input the feature maps of ALL preceding layers (via concatenation, not summation), and passes its own output to all subsequent layers. This means features learned early (like edges, spots, colour patterns on a leaf) are directly reused deeper in the network instead of being re-learned.
3. **Benefit for limited data:** Because features are reused rather than relearned, DenseNet requires fewer parameters than a plain CNN of similar depth, reduces the risk of overfitting, improves gradient flow (mitigating the vanishing gradient problem), and strengthens feature propagation - all of which make it well-suited for the agricultural organization's limited-sample dataset.
4. **Classification head:** After the final dense block, Global Average Pooling is applied, followed by a fully connected layer with 5 output units (one per disease type) and a Softmax activation to output the predicted disease class.

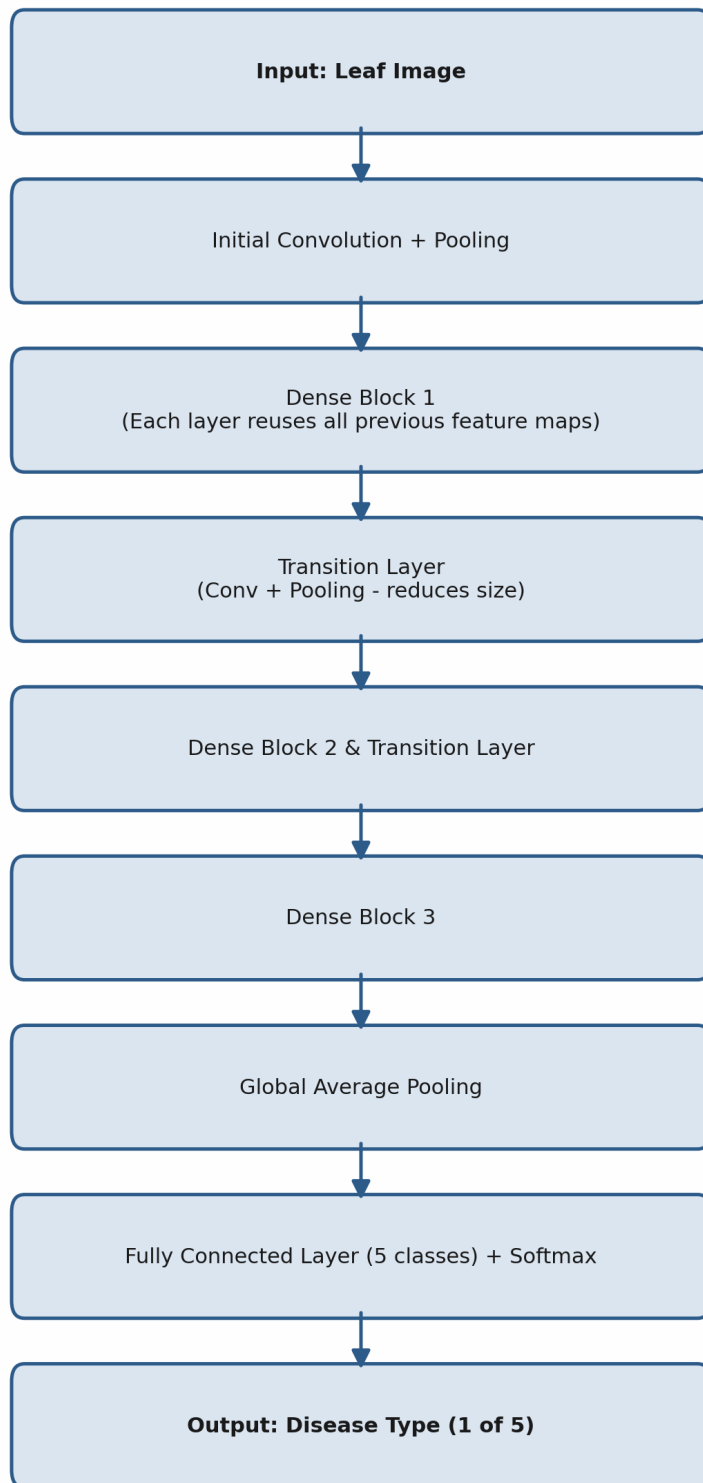


Fig 2: Flowchart - DenseNet-based Plant Disease Classification Model

Question 3

Design an LSTM-based model to predict a student's future academic performance category using weekly attendance, assignment marks, internal assessment marks, and previous examination scores collected over several weeks.

Answer 3

Approach: Since the data (attendance, assignment marks, internal marks, exam scores) is collected weekly and has a natural time-order, an LSTM (Long Short-Term Memory) network is used, as it is specifically designed to learn patterns and dependencies in sequential/time-series data.

1. **Input representation:** For each week, a feature vector is formed containing that week's attendance percentage, assignment marks, internal assessment marks, and (where available) exam scores. The sequence of these weekly vectors over several weeks forms the input sequence to the model.
2. **LSTM layer:** The sequence is fed into one or more LSTM layers. The LSTM's memory cell and gates (input, forget, output gates) allow it to retain important long-term trends (e.g., a consistent drop in attendance) while forgetting irrelevant fluctuations, capturing the temporal dependency across weeks.
3. **Output stage:** The final hidden state of the LSTM (which summarizes the entire weekly sequence) is passed to a fully connected (Dense) layer, followed by a Softmax layer that outputs probabilities over the performance categories (e.g., Excellent, Good, Average, Poor).
4. **Training:** The model is trained using categorical cross-entropy loss, with techniques like dropout between LSTM layers to prevent overfitting, since the model is learning from a limited number of students' weekly records.

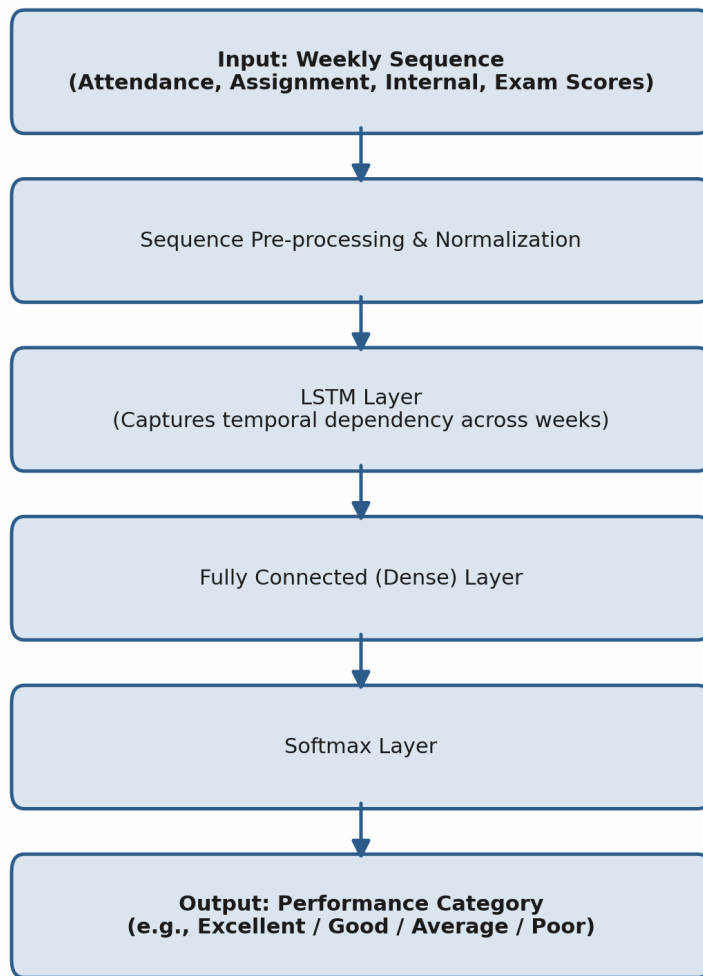


Fig 3: Flowchart - LSTM-based Student Performance Prediction Model

Question 4

Design a Bidirectional RNN model to classify customer reviews as positive, negative, or neutral, and identify the major processing stages from input text to final sentiment prediction.

Answer 4

Approach: Since the meaning of a word in a review can depend on words that come both before and after it (context from both directions), a Bidirectional RNN (BiRNN) is used, which processes the sentence in both the forward and backward directions simultaneously.

Major processing stages:

- 1. Text Pre-processing: The raw review text is cleaned (removing punctuation/special characters), tokenized into words, and converted into numerical form using an Embedding layer, which maps each word to a dense vector capturing semantic meaning.
- 2. Bidirectional RNN Layer: The embedded word sequence is passed through two RNN (or LSTM/GRU) layers - a Forward RNN that reads the sentence left-to-right, and a Backward RNN that reads it right-to-left. This allows each word's representation to be influenced by both past and future context.
- 3. Concatenation: At each time step (or at the final step), the forward and backward hidden states are concatenated to form a combined representation that captures the full bidirectional context.
- 4. Fully Connected Layer: The combined representation is passed through one or more Dense layers to learn higher-level combinations of the extracted features.
- 5. Softmax Output Layer: A final Softmax layer outputs probabilities over the three sentiment classes - Positive, Negative, and Neutral - and the class with the highest probability is selected as the prediction.

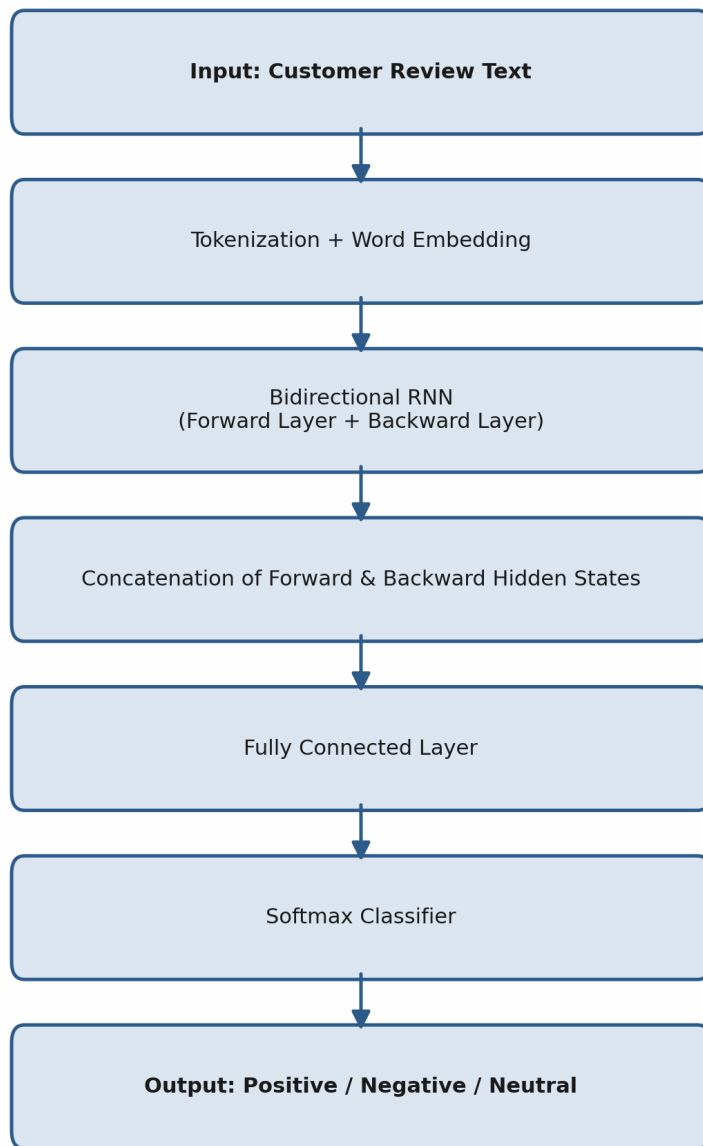


Fig 4: Flowchart - Bidirectional RNN Model for Sentiment Classification

Question 5

Design an Encoder-Decoder model to translate English sentences into Hindi, describing the role of the encoder, decoder, hidden representation, and output layer.

Answer 5

Approach: Since the input (English) and output (Hindi) sentences can have different lengths, and the system must understand the entire input sentence before producing the translation, an Encoder-Decoder (Sequence-to-Sequence) architecture is used.

1. **Encoder:** The Encoder (typically an LSTM/GRU-based RNN) reads the input English sentence word by word. At each time step it updates its internal hidden state, and after processing the entire sentence, its final hidden state summarizes the complete meaning of the input sentence.
2. **Hidden Representation (Context Vector):** The encoder's final hidden state acts as a fixed-length Context Vector (hidden representation) that encodes the overall meaning of the entire input sentence. This vector is passed on as the initial state for the decoder, effectively transferring 'understanding' of the full sentence before any translation begins.
3. **Decoder:** The Decoder (also an LSTM/GRU-based RNN) takes the context vector as its starting hidden state and generates the output Hindi sentence one word at a time. At each step, it uses its current hidden state (and the previously generated word) to predict the next word, continuing until an end-of-sentence token is generated.
4. **Output Layer:** At each decoding time step, a Softmax output layer converts the decoder's hidden state into a probability distribution over the entire Hindi vocabulary, and the word with the highest probability is chosen as the next output word.
5. **Handling variable lengths:** Because the encoder compresses the whole input into a single context vector and the decoder generates output independently, step by step, the model naturally supports input and output sentences of different lengths - a key requirement for English-to-Hindi translation.

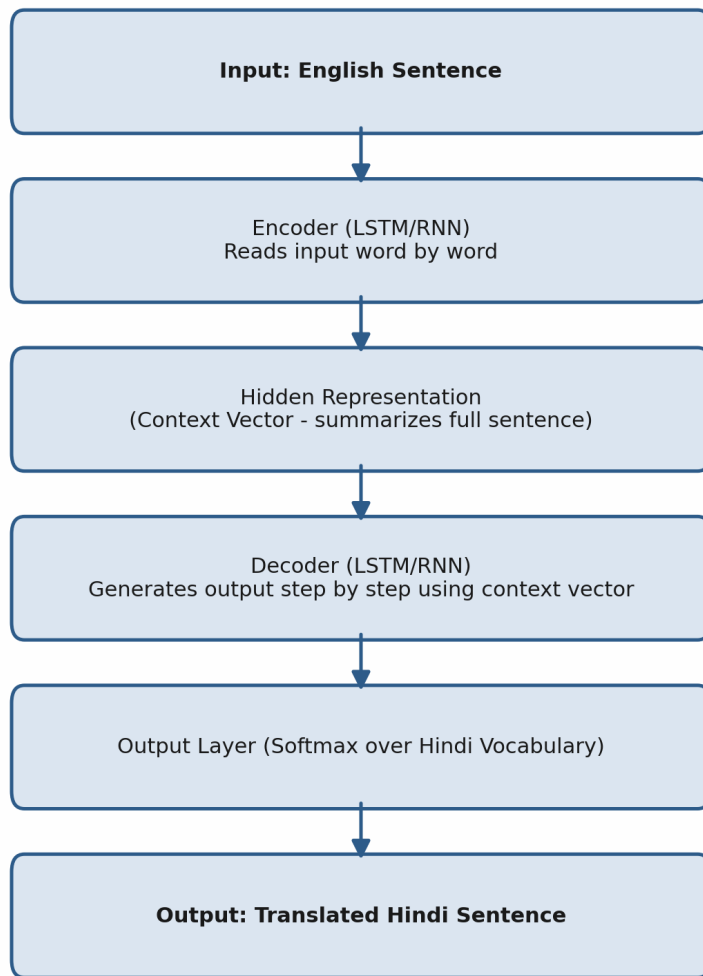


Fig 5: Flowchart - Encoder-Decoder Model for English-to-Hindi Translation